

SQL

Subquery, groupby, having, joins, constraints , foreign key

```
-- subquery

CREATE TABLE employee(
    emp_id INT,
    emp_name VARCHAR(50),
    salary INT,
    department VARCHAR(20)
)

INSERT INTO employee VALUES(1, 'Arun', 30000, 'IT'),
(2, "Divya", 45000, "HR"),
(3, "Karthik", 50000, "IT"),
(4, "Meena", 35000, "Finance"),
(5, "Rahul", 60000, "IT");

-- DQL

SELECT * FROM employee;

SELECT MAX(salary) FROM employee;

SELECT AVG(salary) FROM employee;

-- subquery

SELECT emp_name, salary
FROM employee
WHERE salary > (
    SELECT AVG(salary) FROM employee
)

SELECT emp_name, salary
FROM employee
WHERE salary > (
    SELECT AVG(salary) FROM employee
)

SELECT emp_name FROM employee
WHERE department IN ("IT", "HR")
```

```
SELECT emp_name, salary FROM employee
WHERE salary > ANY (
    SELECT salary
    FROM employee
    WHERE department = "IT"
)
```

```
SELECT emp_name, salary FROM employee
WHERE salary > ALL (
    SELECT salary
    FROM employee
    WHERE department = "IT"
)
```

```
SELECT emp_name, (SELECT AVG(salary) FROM employee) AS avg_salary
FROM employee;
```

```
SELECT department, avg_sal
FROM (
    SELECT department , AVG(salary) AS avg_sal
    FROM employee
    GROUP BY department
)AS dept_avg;
```

```
CREATE TABLE departments(
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL,
    dept_location VARCHAR(50)
)
```

```
CREATE TABLE students(
    stu_id INT PRIMARY KEY ,
    stu_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    salary DECIMAL(10,2) CHECK (salary >0),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
)
```

```
INSERT INTO departments VALUES(1,"HR", "Chennai"),
(2,"IT","Bengaluru"),
```

```

(3, "Finance", "Delhi");

INSERT INTO students VALUES(101, "Pirithika", "pirithika@gmail.com", 50000,
2),
(102, "Suji", "suji@gmail.com", 40000, 1),
(103, "priya", "priya@gmai.com", 70000, 3),
(104, "viji", "viji@gmail.com", 30000, 1),
(105, "maha", "maha@gmail.com", 100000, 2),
(106, "kaviya", "kaviya@gmai.com", 60000, 3);

SELECT s.stu_name, d.dept_name
FROM students s
INNER JOIN departments d ON s.dept_id = d.dept_id;

INSERT INTO students VALUES(107, "Raj", "raj@gmail.com", 50000, NULL)

SELECT s.stu_name, d.dept_name
FROM students s
LEFT JOIN departments d ON s.dept_id = d.dept_id;

SELECT s.stu_name, d.dept_name
FROM students s
RIGHT JOIN departments d ON s.dept_id = d.dept_id;

SELECT s.stu_name, d.dept_name
FROM students s
FULL JOIN departments d ON s.dept_id = d.dept_id;

SELECT s.stu_name, d.dept_name
FROM students s
CROSS JOIN departments d
-- select department which has more than 2 employee
SELECT d.dept_name, COUNT(d.dept_name) AS total_employee
FROM students s
JOIN departments d ON d.dept_id = s.dept_id
GROUP BY d.dept_name
HAVING COUNT(d.dept_name) >2;

INSERT INTO students VALUES(108, "naren", "naren@gmail.com", 80000, 2)

```

Theory Notes

1. Joins

- **INNER JOIN** → returns rows with matching values in both tables.
- **LEFT JOIN** → returns all rows from left table + matching rows from right.
- **RIGHT JOIN** → returns all rows from right table + matching rows from left.
- **FULL JOIN** → returns all rows when there is a match in either table (not supported in MySQL directly, but can be simulated with UNION).
- **CROSS JOIN** → returns Cartesian product (all combinations).

2. Aggregate Functions

- **COUNT()** → counts rows.
- **SUM()** → adds values.
- **AVG()** → calculates average.
- **MAX()** → highest value.
- **MIN()** → lowest value.

3. GROUP BY and HAVING

- **GROUP BY** → groups rows based on column values.
- **HAVING** → filters groups (like WHERE but for aggregates).
- Example:

```
SELECT dept_name, COUNT(*) AS total  
FROM students s  
JOIN departments d ON s.dept_id = d.dept_id  
GROUP BY dept_name  
HAVING COUNT(*) > 2;
```

4. Subqueries

- **Definition:** Query inside another query.
- Types:

- **Single-row subquery** → returns one value.
- **Multi-row subquery** → returns multiple values.
- **Correlated subquery** → depends on outer query.
- Operators:
 - IN → matches any value in subquery.
 - ANY → compares with any value in subquery.
 - ALL → compares with all values in subquery.

5. Constraints Recap

- PRIMARY KEY → unique + not null.
- UNIQUE → ensures no duplicate values.
- CHECK → enforces condition.
- FOREIGN KEY → links tables.
- NOT NULL → prevents null values.

Short Answer

1. How does INNER JOIN differ from LEFT JOIN?
2. What is the purpose of FOREIGN KEY?
3. When would you use ANY vs ALL in subqueries?

Coding Exercises

1. Write a query to list all students with their department names.
2. Find departments with more than 2 employees.
3. Select employees whose salary is greater than the average salary.
4. Use a subquery to find employees in IT with salary higher than any HR employee.
5. Write a query to get the maximum salary per department.
6. Demonstrate a cross join between students and departments.
7. Create a subquery that returns department-wise average salary and use it in the outer query.